

PPVSSlurm-Final Report Draft

Author1

Author2

Author3

Author4

UNI
institute
chair

Supervisor: SupervisorName

March 5, 2026

Contents

1	Introduction and Main Goal	2
2	How We Approached the Project	2
2.1	Execution Strategy	2
2.2	Main Constraints	2
3	Current Code Structure (Overview)	3
3.1	Core Scripts	3
3.2	Backend Modes	3
3.3	Profile-Driven Portability	3
4	Submission Workflow	3
4.1	Preparation	3
4.2	User Submission Options	4
4.3	What Is Produced per Run	4
5	Migration: Azure to TU-HPC	5
5.1	Azure Phase	5
5.2	TU-HPC Phase	5
6	Reference-Scale Workload	5
6.1	Target Combination Space	5
6.2	Data Inputs	5
7	What Data We Gather and Why	5
8	Observability Overview	6
9	Validation Status and Open Risks	6
10	Conclusion	6
	Appendix A: Key Commands	8
	AI Assistance Disclosure	9

1 Introduction and Main Goal

In this project, we built a practical SLURM workflow for Vessim simulations [1, 2]. Our main goal was to move from manual and cluster-specific execution to a reproducible operator flow that users can run with one command.

We focused on four outcomes:

- reliable parameter sweeps via SLURM arrays,
- deterministic scenario generation via Hydra,
- consistent output artifacts per task,
- portability across cluster environments.

We started our setup work experimentally on an Azure student environment and then migrated the same workflow to the managed TU-HPC environment.

2 How We Approached the Project

2.1 Execution Strategy

We decided early that the workflow should not depend on one specific cluster layout. Therefore, we designed one main entrypoint and moved environment differences into profile variables.

The final operator chain is:

1. Hydra generates scenario YAML files from override inputs.
2. A selector CSV maps one scenario per row.
3. A SLURM array executes one task per row.

This lets us scale from smoke tests to large sweeps without changing the basic user flow.

2.2 Main Constraints

During implementation, we had to work with:

- managed HPC permissions (no sudo on TU-HPC),
- shared filesystem visibility between login and compute nodes,
- SLURM limits such as `MaxArraySize`,
- cloud availability constraints on the Azure student setup.

3 Current Code Structure (Overview)

3.1 Core Scripts

The project is structured around a small set of scripts with clear responsibilities:

- `experiments/hydra_gen/run_hybrid.sh`: main one-command entrypoint.
- `experiments/hydra_gen/generate_scenarios.py`: Hydra multirun generation.
- `experiments/hydra_gen/build_csv.py`: selector CSV creation.
- `experiments/sbatch/vessim_sweep_array.sbatch`: SLURM array wrapper.
- `experiments/run_scenario.py`: default large simulation backend.
- `experiments/simple/run_scenario_simple.py`: lightweight backend.
- `experiments/setup_env.sh`: environment validation before submit.
- `experiments/bootstrap/*`: runtime bootstrap, preflight, Python setup.

3.2 Backend Modes

We currently support two execution modes:

- **main** (default): SAM-based backend for wind/solar/battery capacity studies.
- **simple**: lightweight actor/battery simulations for quick checks.

The mode can be switched with `PPVS_MODE=main|simple`.

3.3 Profile-Driven Portability

Cluster-specific behavior is configured via profile variables (paths, SLURM routing, dataset naming). We use:

- runtime path variables (`PPVS_ROOT`, `PPVS_PARAMS_DIR`, `PPVS_RUNS_DIR`, `PPVS_RESULTS_DIR`),
- SLURM routing variables (`SLURM_PARTITION`, `SLURM_ACCOUNT`, `SLURM_QOS`),
- data-profile variables (`PPVS_MAIN_DATA_PROFILE=generic|reference` and per-file overrides).

4 Submission Workflow

4.1 Preparation

Before a production run, we use the same sequence:

1. preflight checks,

2. Python environment installation/update,
3. setup validation.

In `main` mode, this includes required dataset checks and import checks (including PySAM).

4.2 User Submission Options

We support multiple submission inputs while keeping the same operator:

1. CLI override submission:

```
./experiments/hydra_gen/run_hybrid.sh \
  --profile <profile.env> \
  key=value[,value2] ...
```

2. overrides from file:

```
./experiments/hydra_gen/run_hybrid.sh \
  --profile <profile.env> \
  --overrides-file my_overrides.txt
```

3. simple mode submission:

```
PPVS_MODE=simple ./experiments/hydra_gen/run_hybrid.sh \
  --profile <profile.env> \
  ...
```

4.3 What Is Produced per Run

For each submission, we generate:

- scenario YAML folder (`params/scenarios/generated_<timestamp>/`),
- selector CSV (`params/scenario_selector_<timestamp>.csv`),
- scheduler logs (`results/%x_%A_%a.out/.err`),
- per-task run folder (`runs/<jobid>/<task>_<scenario_id>/...`).

Per task, we collect at least:

- `results.csv`,
- `meta.json`,
- `summary_event.json`,
- `merged_data.csv` (main mode).

5 Migration: Azure to TU-HPC

5.1 Azure Phase

We started on Azure as an experimental base cluster. This phase was useful to validate the full loop (submission, scheduling, artifacts, accounting) with a controller/compute separation and shared filesystem.

5.2 TU-HPC Phase

We then moved to TU-HPC with no admin privileges and strict managed-cluster constraints. The important part is that our user-facing submission flow stayed the same, because environment-specific details are externalized in the profile.

On TU-HPC, `MaxArraySize=1001` is lower than our largest target sweep (1089 tasks). We addressed this in the operator by automatic chunking plus `ARRAY_OFFSET` mapping, so users do not need to split selector files manually.

6 Reference-Scale Workload

6.1 Target Combination Space

For the reference-scale experiment [3], we target:

- wind capacities: 11 values,
- solar capacities: 11 values,
- battery capacities: 9 values,
- total: $11 \times 11 \times 9 = 1089$ combinations.

Baseline simulation parameters:

- `sim_start` = 2020-01-01 00:00:00,
- `step_size_s` = 60,
- `until_s` = 31536000.

6.2 Data Inputs

In main mode, we require load, wind, solar, turbine, and carbon inputs. The code supports generic names and reference-compatible names. This makes the workflow reusable while still allowing compatibility with established datasets.

7 What Data We Gather and Why

During runs, we gather technical data at three levels:

1. **Simulation output data:** time-series results and merged carbon/coverage data (`results.csv`, `merged_data.csv`).
2. **Run metadata:** scenario parameters, runtime context, and SLURM metadata (`meta.json`, `summary_event.json`).
3. **Scheduler/accounting data:** job state, elapsed time, requested/used resources via `sacct`.

This separation helps us validate correctness, performance, and reproducibility.

8 Observability Overview

Besides standard SLURM commands (`squeue`, `scontrol`, `sacct`), we implemented an optional observability bridge:

- `collect_sacct.py` exports accounting records,
- `collect_runs.py` exports per-run summary events,
- `push_ndjson.py` pushes NDJSON to Elasticsearch.

For self-managed environments, we also provide an optional local stack bootstrap script. For managed HPC environments, users can export and push to an external observability backend.

9 Validation Status and Open Risks

We validated the workflow in both cluster contexts with the same operator flow. The main unresolved operational risk is infrastructure availability on the cloud side (subscription lifecycle), which can temporarily block retests.

For reproducibility, we should always archive:

- exact override inputs,
- generated scenarios and selector CSV,
- run outputs and scheduler logs,
- the exact input dataset snapshot.

10 Conclusion

In summary, we implemented a practical and reproducible SLURM workflow for Vessim that our group can run on different cluster environments with minimal changes.

The most important results are:

- one consistent submission operator,
- support for both quick tests and large reference-scale sweeps,
- stable artifact layout per task,

- optional observability export for deeper analysis.

This gives us a solid technical basis for final experiments, report integration, and future extensions.

Appendix A: Key Commands

Main mode submission

```
./experiments/hydra_gen/run_hybrid.sh --profile experiments/profiles/mycluster.env \  
wind_system_capacity=0,3000,6000 \  
solar_system_capacity=0,4000,8000 \  
battery_capacity=0,7500,15000 \  
step_size_s=60 until_s=86400
```

Simple mode submission

```
PPVS_MODE=simple ./experiments/hydra_gen/run_hybrid.sh \  
--profile experiments/profiles/mycluster.env \  
battery.capacity_wh=0,100,300 actors.1.signal.value=1000,2000 \  
step_size_s=60 until_s=3600
```

Overrides from file

```
./experiments/hydra_gen/run_hybrid.sh \  
--profile experiments/profiles/mycluster.env \  
--overrides-file my_overrides.txt
```

AI Assistance Disclosure

We used OpenAI-based AI tools to support language editing, documentation drafting, information lookup, and scaffolding of non-creative technical artifacts (e.g., wrapper/shell script structure). Due to limited time and compute resources, AI support was used to accelerate repetitive tasks. All AI outputs were treated as suggestions and were manually reviewed, tested, and corrected by the authors before inclusion. Final technical decisions, experiment setup, results, and conclusions remain the responsibility of the authors.

References

- [1] P. Wiesner et al., *Vessim: A Testbed for Carbon-Aware Applications and Systems*, HotCarbon 2024.
- [2] P. Wiesner et al., *Software-in-the-Loop Simulation for Developing and Testing Carbon-Aware Applications*, Software: Practice and Experience, 2023.
- [3] J. Irion et al., *Optimizing Microgrid Composition for Sustainable Data Centers*, DOI: 10.1145/3731599.3767562.
- [4] SchedMD, *SLURM Workload Manager Documentation*, <https://slurm.schedmd.com/documentation.html>.